

Chapter 2. Performance Testing Methodology

Performance testing is undertaken for a variety of reasons. In this chapter, we will introduce the different types of performance test that a team may wish to execute and discuss some best practices for each subtype of testing.

Later in the chapter, we will discuss statistics—and some very important human factors—that are often neglected when considering performance problems.

Types of Performance Tests

Performance tests are frequently conducted for the wrong reasons, or conducted badly. The reasons for this vary widely but are often rooted in a failure to understand the nature of performance analysis and a belief that “doing something is better than doing nothing.” As we will see several times throughout the book, this belief is often a dangerous half-truth at best.

One of the more common mistakes is to speak generally of “performance testing” without engaging with the specifics. In fact, there are many different types of large-scale performance tests that can be conducted on a system.

NOTE

Good performance tests are quantitative. They ask questions that produce a numeric answer that can be handled as an experimental output and subjected to statistical analysis.

The types of performance tests we will discuss in this book usually have independent (but somewhat overlapping) goals. It is, therefore, important to understand the quantitative questions you are trying to answer before deciding what type of testing should be carried out.

This doesn't have to be that complex—simply writing down the questions the test is intended to answer can be enough. However, it is usual to consider why these tests are important for the application and confirming the reason with the application owner (or key customers).

Some of the most common test types, and an example question for each, are as follows:

Latency test

What is the end-to-end transaction time?

Throughput test

How many concurrent transactions can the current system capacity deal with?

Stress test

What is the breaking point of the system?

Load test

Can the system handle a specific load?

Endurance test

What performance anomalies are discovered when the system is run for an extended period?

Capacity planning test

Does the system scale as expected when additional resources are added?

Degradation

What happens when the system is partially failed?

Let's look in more detail at each of these test types in turn.

Latency Test

Latency is one of the most common types of performance test because it is often a system observable of keen interest to management (and users): how long are our customers waiting for a transaction (or a page load)?

This can be a double-edged sword because the simplicity of the question (that a latency test seeks to answer) can cause teams to focus too much on

latency. This, in turn, can cause the team to ignore the necessity of identifying quantitative questions for other types of performance tests.

NOTE

The goal of a latency tuning exercise is usually to directly improve the user experience or to meet a service-level agreement.

However, even in the simplest cases, a latency test has some subtleties that must be treated carefully. One of the most noticeable is that a simple mean (average) is not very useful as a measure of how well an application is reacting to requests. We will discuss this subject more fully in [“Statistics for JVM Performance”](#) and explore additional measures.

Throughput Test

Throughput is probably the second most common quantity to be performance tested. It can even be thought of as dual to latency, in some senses.

For example, when we are conducting a latency test, it is important to state (and control) the concurrent transactions count when producing a distribution of latency results. Similarly, when we are conducting a throughput test, we must make sure to keep an eye on latency and check that it is not blowing up to unacceptable values as we ramp up.

NOTE

The observed latency of a system should be stated at known and controlled throughput levels, and vice versa.

We determine the “maximum throughput” by noticing when the latency distribution suddenly changes—effectively a “breaking point” (also called an *inflection point*) of the system. The point of a stress test, as we will see in an upcoming section, is to locate such points and the load levels at which they occur.

A throughput test, on the other hand, is about measuring the observed maximum throughput before the system starts to degrade. Once again, these test types are discussed separately but are rarely truly independent in practice.

Stress Test

One way to think about a stress test is as a way to determine how much spare headroom the system has. The test typically proceeds by placing the system into a steady state of transactions—that is, a specified throughput level (often the current peak). The test then ramps up the concurrent transactions slowly, until the system observables start to degrade.

The value just before the observables started to degrade determines the maximum throughput achieved in a stress test.

Load Test

A load test differs from a throughput test (or a stress test) in that it is usually framed as a binary test: “Can the system handle this projected load or not?” Load tests are sometimes conducted in advance of expected business events—for example, the onboarding of a new customer or market that is expected to drive greatly increased traffic to the application.

Other examples of possible events that could warrant performing this type of test include advertising campaigns, social media events, and “viral content.”

Endurance Test

Some problems manifest only over much longer periods of time (often measured in days). These include slow memory leaks, cache pollution, and memory fragmentation (especially for applications that may eventually suffer a GC concurrent mode failure; see [Chapter 5](#) for more details).

To detect these types of issues, an endurance test (also known as a soak test) is the usual approach. These are run at average (or high) utilization, but within observed realistic loads for the system. During the test, resource levels are closely monitored to spot any breakdowns or exhaustions of resources.

This type of test is more common in low-latency systems, as it is very common that those systems will not be able to tolerate the length of a stop-the-world event caused by a full GC cycle (see [Chapter 4](#) and subsequent chapters for more on stop-the-world events and related GC concepts).

Endurance tests are not performed as often as they perhaps should be, for the simple reason that they take a long time to run and can be very expensive—but there are no shortcuts. There is also the inherent difficulty of testing with realistic data or usage patterns over a long period. This can be one of the major reasons why teams end up “testing in production.”

This type of test is also not always applicable to microservice or other architectures where a lot of code changes may be deployed in a short time.

Capacity Planning Test

Capacity planning tests bear many similarities to stress tests, but they are a distinct type of test. The role of a stress test is to find out what the current system will cope with, whereas a capacity planning test is more forward-looking and seeks to find out what load an upgraded system could handle.

For this reason, capacity planning tests are often carried out as part of a scheduled planning exercise rather than in response to a specific event or threat.

Degradation Test

Once upon a time, rigorous failover and recovery testing was really only practiced in the most highly regulated and scrutinized environments (including banks and financial institutions). However, as applications have migrated to the cloud, clustered deployments (e.g., based on Kubernetes) have become more common. One primary consequence of this is that more developers now need to be aware of the possible failure modes of clustered applications.

NOTE

A full discussion of all aspects of resilience and failover testing is outside the scope of this book. In [Chapter 14](#), we will discuss some of the simpler effects that can be seen in cloud systems when a cluster partially fails or needs to recover.

In this section, the only type of resilience test we will discuss is the degradation test—this type of test is also known as a *partial failure* test.

The basic approach to this test is to see how the system behaves when a component or entire subsystem suddenly loses capacity while the system is running at simulated loads equivalent to usual production volumes. Examples could be application server clusters that suddenly lose members or network bandwidth that suddenly drops.

Key observables during a degradation test include the transaction latency distribution and throughput.

One particularly interesting subtype of partial failure test is known as the *[Chaos Monkey](#)*. This is named after a project at Netflix that was undertaken to verify the robustness of its infrastructure.

The idea behind Chaos Monkey is that in a truly resilient architecture, the failure of a single component should not be able to cause a cascading failure or have a meaningful impact on the overall system.

Chaos Monkey forces system operators to confront this possibility by randomly killing off live processes in the production environment.

To successfully implement Chaos Monkey-type systems, an organization must have very high levels of system hygiene, service design, and operational excellence. Nevertheless, it is an area of interest and aspiration for an increasing number of companies and teams.

As we will see several times in this book, well-architected cloud systems are “designed for failure,” in the sense that in a large enough deployment, some aspect of the system is broken at any given time. Perfectly working systems are an artifact of small system sizes, not the norm.

In addition, many systems are built with few assumptions about the particulars of hardware and infrastructure. These systems may also choose not to couple directly to specific features provided by their cloud vendor—this means that it is possible to run on top of multiple platforms for redundancy and failover reasons.

In this case, the software design accounts for, and encodes, these failure assumptions. As a result, coding for cloud can look very different from writing code to run on large upfront investments of static infrastructure—which are designed to “minimally fail.”

Best Practices Primer

When deciding where to focus your effort in a performance tuning exercise, three golden rules can provide useful guidance:

- Identify what you care about and figure out how to measure it.
- Optimize what matters, not what is easy to optimize.
- Start by optimizing the largest contributions.

The second point has a converse, which is to remind yourself not to fall into the trap of attaching too much significance to whatever quantity you can easily measure. Not every observable is significant to a business, but it is sometimes tempting to report on an easy measure rather than the right measure.

To the third point, it is also easy to fall into the trap of optimizing small things simply for the sake of optimizing.

Top-Down Performance

One of the aspects of Java performance that many engineers miss at first sight is that large-scale benchmarking of Java applications is usually much easier than trying to get accurate numbers for small sections of code.

This is such a widely misunderstood point that, to deliberately deemphasize it, we do not discuss *microbenchmarking* in the main book text at all. Instead, it is discussed in [Appendix A](#), a placement that more accurately reflects the utility of the technique for the majority of applications.

NOTE

The approach of starting with the performance behavior of an entire application is usually called *top-down* performance.

To make the most of the top-down approach, a testing team needs a test environment, a clear understanding of what it needs to measure and optimize, and an understanding of how the performance exercise will fit into the overall software development lifecycle.

Creating a Test Environment

Setting up a test environment is one of the first tasks most performance testing teams will need to undertake. Wherever possible, this should be an exact duplicate of the production environment in all aspects.

NOTE

Some teams may be in a position where they are forced to forgo testing environments and simply measure in production using modern deployment and observability techniques. This is the subject of [Chapter 10](#), but it is not recommended as an approach unless it's necessary.

This includes not only application servers (which should have the same number of CPUs, same version of the OS and Java runtime, etc.) but also web servers, databases, message queues, and so on. Any services (e.g., third-party network services that are not easy to replicate or do not have sufficient QA capacity to handle a production-equivalent load) will need to be mocked for a representative performance testing environment.

NOTE

Performance testing environments that are significantly different from the production deployments that they purport to represent are usually ineffective—they fail to produce results that have any usefulness or predictive power in the live environment.

For traditional (i.e., non-cloud-based) environments, a production-like performance testing environment is relatively straightforward to achieve in theory—the team simply buys as many machines as are in use in the production environment and then configures them in exactly the same way as production is configured.

Management is sometimes resistant to the additional infrastructure cost this represents. This is almost always a false economy, but sadly, many organizations fail to account correctly for the cost of outages. This can lead to a belief that the savings from not having an accurate performance testing environment are meaningful, as it fails to properly account for the risks introduced by having a QA environment that does not mirror production.

The advent of cloud technologies has changed this picture. More dynamic approaches to infrastructure management are now widespread. This includes on-demand and autoscaling infrastructure, as well as approaches such as *immutable infrastructure*, also referred to as treating server infrastructure as “livestock, not pets.”

In theory, these trends make the construction of a performance testing environment that looks like production easier. However, there are subtleties here. For example:

- Having a process that allows changes to be made in a test environment first and then migrated to production
- Making sure that a test environment does not have some overlooked dependencies that depend upon production
- Ensuring that test environments have realistic authentication and authorization systems, not dummy components

Despite these concerns, the possibility of setting up a testing environment that can be turned off when not in use is a key advantage of cloud-based deployments. This can bring significant cost savings to the project, but it requires a proper process for starting up and shutting down the environment as scheduled.

Identifying Performance Requirements

The overall performance of a system is not solely determined by your application code. As we will discover throughout the rest of this book, the container, operating system, and hardware all have a role to play.

NOTE

In [“A Simple System Model”](#), we will meet a simple system model that describes in more detail how the interaction between OS, hardware, JVM, and code impacts performance.

Therefore, the metrics that we will use to evaluate performance should not be thought about solely in terms of the code. Instead, we must consider systems as a whole and the observable quantities that are important to customers and management. These are usually referred to as performance *nonfunctional requirements* (NFRs) and are the key indicators that we want to optimize.

NOTE

One useful approach is that of service level objectives (SLOs), which we will discuss in [Chapter 10](#).

Some performance goals are obvious:

- Reduce 95% percentile transaction time by 100 ms.
- Improve system so that 5x throughput on existing hardware is possible.
- Improve average response time by 30%.

Others may be less apparent:

- Reduce resource cost to serve the average customer by 50%.
- Ensure system is still within 25% of response targets, even when application clusters are degraded by 50%.
- Reduce customer “drop-off” rate by 25% by removing 10 ms of latency.

An open discussion with the stakeholders as to exactly what should be measured and what goals are to be achieved is essential. Ideally, this discussion should form part of the first kickoff meeting for any performance exercise.

Performance Testing as Part of the SDLC

Some companies and teams prefer to think of performance testing as an occasional, one-off activity. However, more sophisticated teams tend to make ongoing performance tests, and in particular, performance regression testing, an integral part of their software development lifecycle (SDLC).

This requires collaboration between developers and infrastructure teams to control which versions of code are present in the performance testing environment at any given time. It is also virtually impossible to implement without a dedicated testing environment.

Java-Specific Issues

Much of the science of performance analysis is applicable to any modern software system. However, the nature of the JVM is such that there are certain additional complications that the performance engineer should be aware of and consider carefully. These largely stem from the dynamic

self-management capabilities of the JVM, such as the dynamic tuning of memory areas and JIT compilation.

For example, modern JVMs analyze which methods are being run to identify candidates for JIT compilation to optimized machine code. This means that if a method is not being JIT-compiled, then one of two things is true about the method:

- It is not being run frequently enough to warrant being compiled.
- The method is too large or complex to be analyzed for compilation.

The second condition is, by the way, much rarer than the first. In [Chapter 6](#), we will discuss JIT compilation in detail and show some simple techniques for ensuring that the important methods of applications are targeted for JIT compilation by the JVM.

Having discussed some of the most common best practices for performance, let's now turn our attention to the pitfalls and antipatterns that teams can fall prey to.

Causes of Performance Antipatterns

An antipattern is an undesired behavior of a software project or team that is observed across a large number of projects.¹ The frequency of occurrence leads to the conclusion (or suspicion) that some underlying factor is responsible for creating the unwanted behavior. Some antipatterns may at first sight seem to be justified, with their nonideal aspects not immediately obvious. Others are the result of negative project practices slowly accreting over time.

A partial catalog of antipatterns can be found in [Appendix B](#)—where an example of the first kind would be something like *Distracted by Shiny*, whereas *Tuning by Folklore* is an example of the second kind.

In some cases, the behavior may be driven by social or team constraints, or by common misapplied management techniques, or by simple human (and developer) nature. By classifying and categorizing these unwanted features, we develop a *pattern language* for discussing them, and hopefully eliminating them from our projects.

Performance tuning should always be treated as a very objective process, with precise goals set early in the planning phase. This is easier said than

done: when a team is under pressure or not operating under reasonable circumstances, this can simply fall by the wayside.

Many readers will have seen the situation where a new client is going live or a new feature is being launched, and an unexpected outage occurs—in user acceptance testing (UAT) if you are lucky but often in production. The team is then left scrambling to find and fix what has caused the bottleneck. This usually means performance testing has not been carried out, or the team “ninja” made an assumption and has now disappeared (ninjas are good at this).

A team that works in this way will likely fall victim to antipatterns more often than a team that follows good performance testing practices and has open and reasoned conversations. As with many development issues, it is often the human elements, such as communication problems, rather than any technical aspect, that leads to an application having problems.

One interesting possibility for classification was provided in a blog post by Carey Fichel called [“Why Developers Keep Making Bad Technology Choices”](#). The post specifically calls out five main reasons that cause developers to make bad choices. Let’s look at each in turn.

Boredom

Most developers have experienced boredom in a role, and for some, this doesn’t have to last very long before they seek a new challenge or role—either in the company or elsewhere. However, other opportunities may not be present in the organization, and moving somewhere else may not be possible.

It is likely many readers have come across a developer who is simply riding it out, perhaps even actively seeking an easier life. However, bored developers can harm a project in a number of ways.

For example, they might introduce code complexity that is not required, such as writing a sorting algorithm directly in code when a simple `Collections.sort()` would be sufficient. They might also express their boredom by looking to build components with technologies that are unknown or perhaps don’t fit the use case just as an opportunity to use them—which leads us to the next section.

Résumé Padding

Occasionally the overuse of technology is not tied to boredom but rather represents the developer exploiting an opportunity to boost their experience with a particular technology on their résumé (or CV).

In this scenario, the developer is making an active attempt to increase their potential salary and marketability as they're about to re-enter the job market. It's unlikely that many people would get away with this inside a well-functioning team, but it can still be the root of a choice that takes a project down an unnecessary path.

The consequences of an unnecessary technology being added due to a developer's boredom or résumé padding can be far-reaching and very long-lived, lasting for many years after the original developer has left.

Social Pressure

Technical decisions are often at their worst when concerns are not voiced or discussed at the time choices are being made. This can manifest in a few ways; for example, perhaps a junior developer doesn't want to make a mistake in front of more senior members of their team, or perhaps a developer fears appearing to their peers as uninformed on a particular topic.

Another particularly toxic type of social pressure is for competitive teams, wanting to be seen as having high development velocity, to rush key decisions without fully exploring all the consequences.

Lack of Understanding

Developers may look to introduce new tools to help solve a problem because they are not aware of the full capability of their current tools. It is often tempting to turn to a new and exciting technology component because it is great at performing one specific task. However, introducing more technical complexity must be balanced with what the current tools can actually do.

For example, Hibernate is sometimes seen as the answer to simplifying translation between domain objects and databases. If the team has only limited understanding of Hibernate, developers can make assumptions about its suitability based on having seen it used in another project.

This lack of understanding can cause overcomplicated usage of Hibernate and unrecoverable production outages. By contrast, rewriting the entire data layer using simple JDBC calls allows the developer to stay in familiar territory.

One of the authors taught a Hibernate course that contained an attendee in exactly this position; they were trying to learn enough Hibernate to see if the application could be recovered but ended up having to rip out Hibernate over the course of a weekend—definitely not an enviable position.

Misunderstood/Nonexistent Problem

Developers may use a technology to solve a particular issue where the problem space itself has not been adequately investigated. Without having measured performance values, it is almost impossible to understand the success of a particular solution. Often, collating these performance metrics enables a better understanding of the problem.

To avoid antipatterns, it is important to ensure that communication about technical issues is open to all participants in the team and actively encouraged. Where things are unclear, gathering factual evidence and working on prototypes can help to steer team decisions. A technology may look attractive; however, if the prototype does not measure up, then the team can make a more informed decision.

To see how these underlying causes can lead to a variety of performance antipatterns, interested readers should consult [Appendix B](#).

Statistics for JVM Performance

If performance analysis is truly an experimental science, then we will inevitably find ourselves dealing with distributions of results data. Statisticians and scientists know that results that stem from the real world are virtually never represented by clean, stand-out signals. We must deal with the world as we find it, rather than the overidealized state in which we would like to find it.

*In God we trust; all others must use data.*²

—W. Edwards Deming (attr)

All measurements contain some amount of error, so repeated runs must be used to try to minimize the effect of errors in any individual run. The gold standard for Java performance was established in 2007 in the paper [“Statistically Rigorous Java Performance Evaluation”](#).³ This contains the frequently repeated rule that thirty runs are necessary for reasonable statistical behavior in a highly dynamic software system, such as the JVM.

In the next section, we’ll describe the two main types of error that a Java developer can expect to encounter when doing performance analysis.

Types of Error

Two main sources of error an engineer may encounter are:

Random error

A measurement error or an unconnected factor affects results in an uncorrelated.

Systematic error

An unaccounted factor affects measurement of the observable in a correlated way.

Specific words are associated with each type of error. For example, *accuracy* is used to describe the level of systematic error in a measurement; high accuracy corresponds to low systematic error. Similarly, *precision* is the term corresponding to random error; high precision is low random error.

The graphics in [Figure 2-1](#) show the effect of these two types of error on a measurement. The extreme left image shows a clustering of shots (which represent our measurements) around the true result (the center of the target). These measurements have both high precision and high accuracy.

The second image has a systematic effect (miscalibrated sights, perhaps?) that is causing all the shots to be off-target, so these measurements have high precision but low accuracy. The third image shows shots basically on target but loosely clustered around the center, so low precision but high accuracy. The final image shows no clear pattern, a result of having both low precision and low accuracy.

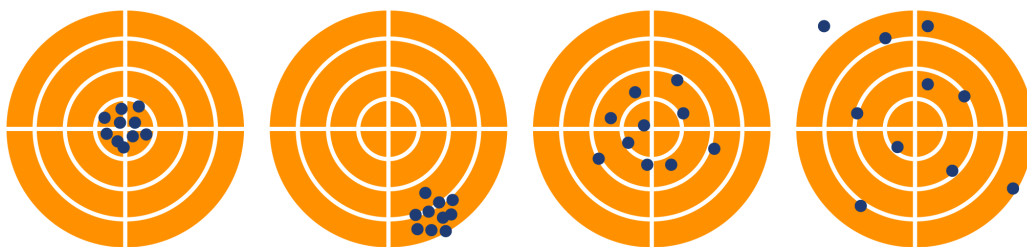


Figure 2-1. Different types of error

Let's move on to explore these types of error in more detail, starting with random error.

Random error

Random errors are hopefully familiar to most people—they are a very well-trodden path. However, they still deserve a mention here, as any handling of observed or experimental data needs to contend with them to some degree.

NOTE

The discussion assumes readers are familiar with basic statistical handling of normally distributed measurements (mean, mode, standard deviation, etc.); readers who aren't should consult a basic textbook, such as the [*Handbook of Biological Statistics*](#).⁴

Random errors are caused by unknown or unpredictable changes in the environment. In general scientific usage, these changes can occur in either the measuring instrument or the environment, but for software, we assume that our measuring harness is reliable, so the source of random error can only be the operating environment.

Random error is usually considered to obey a Gaussian (aka normal) distribution. A couple of typical examples of Gaussian distributions are shown in [Figure 2-2](#).

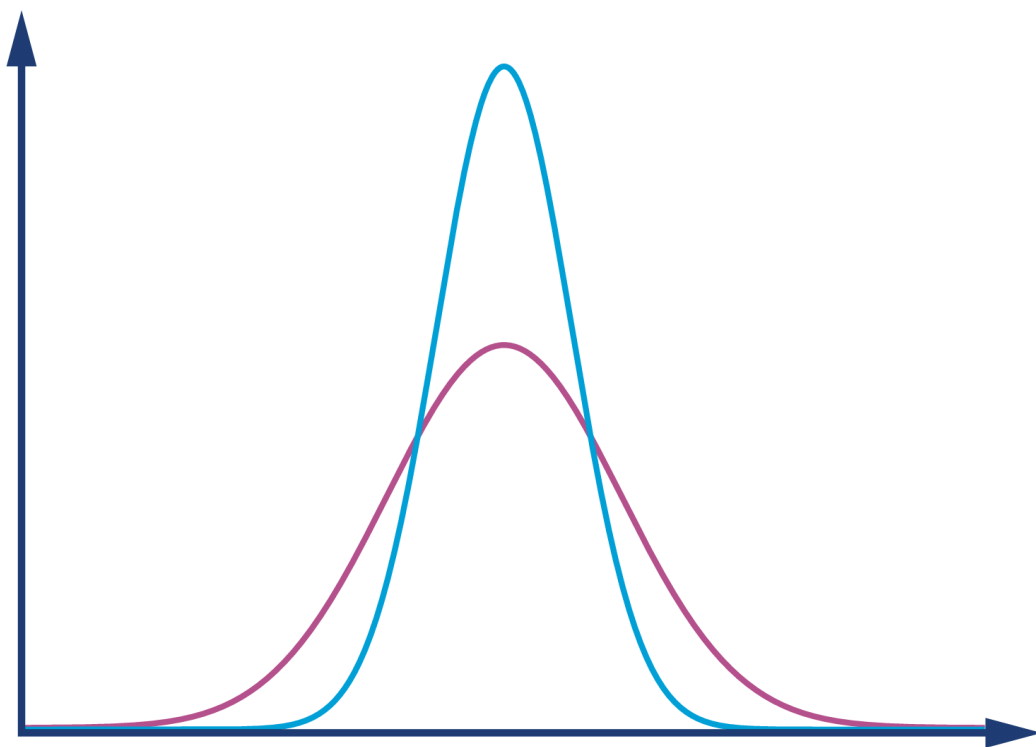


Figure 2-2. A Gaussian distribution (aka normal distribution or bell curve)

The distribution is a good model for the case where an error is equally likely to make a positive or negative contribution to an observable. However, as we will see in the section on non-normal statistics, the situation for JVM measurements is a little more complicated.

Systematic error

As an example of systematic error, consider a performance test running against a group of backend Java web services that send and receive JSON. This type of test is very common when it is problematic to directly use the application frontend for load testing.

[Figure 2-3](#) was generated from the Apache JMeter load-generation tool. In it, there are actually two systematic effects at work. The first is the linear pattern observed in the topmost line (the outlier service), which represents slow exhaustion of some limited server resource.

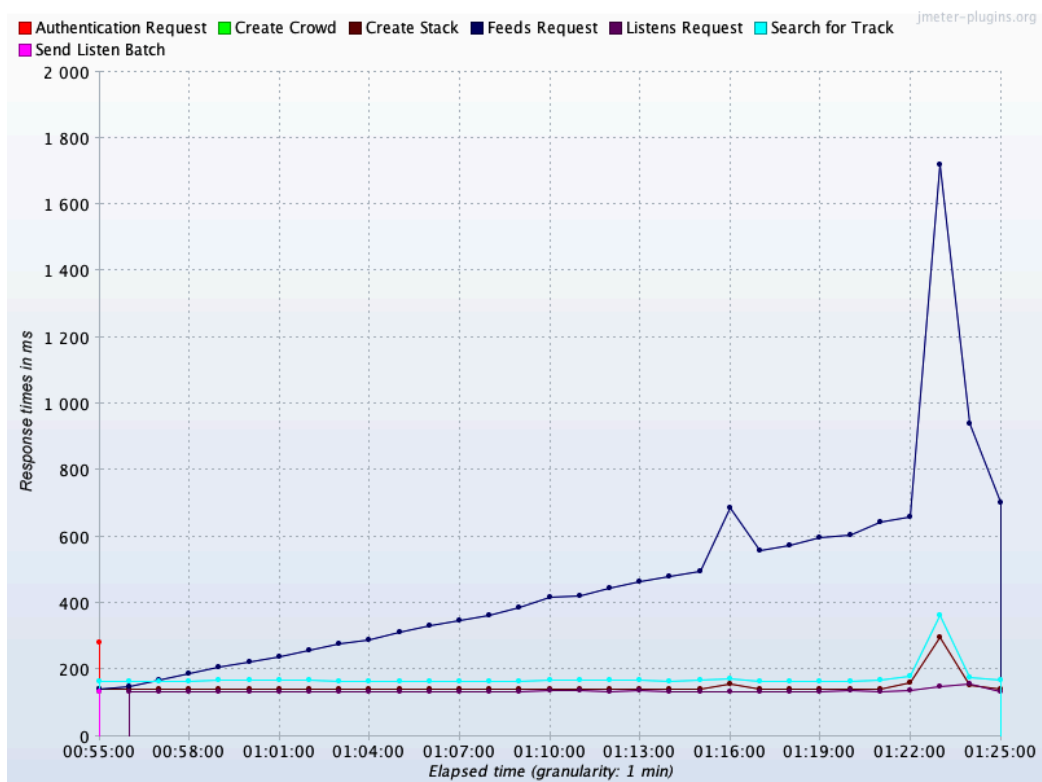


Figure 2-3. Systematic error

This type of pattern is often associated with a memory leak or some other resource being used and not released by a thread during request handling, and it represents a candidate for investigation—this looks like it could be a genuine problem.

NOTE

Further analysis would be needed to confirm the type of resource that was being affected; we can't just conclude that it's a memory leak.

The second effect that should be noticed is the consistency of the majority of the other services at around the 180 ms level. This is suspicious, as the services are doing very different amounts of work in response to a request. So why are the results so consistent?

The answer is that while the services under test are located in London, this load test was conducted from Mumbai, India. The observed response time includes the irreducible round-trip network latency from Mumbai to London. This is in the range of 120–150 ms, so it accounts for the vast majority of the observed time for the services other than the outlier.

This large, systematic effect is drowning out the differences in the actual response time (as the services are actually responding in much less than 120 ms). This is an example of a systematic error that does not represent a problem with our application.

Instead, this error stems from a problem in our test setup not duplicating production, so the good news is that this artifact completely disappeared (as expected) when the test was rerun from London.

We've met some examples of sources of error and mentioned some notorious pitfalls, so let's move on to discuss an aspect of JVM performance measurement that requires some special care and attention to detail.

Non-Normal Statistics

Statistics based on the normal distribution do not require much mathematical sophistication. For this reason, the standard approach to statistics that is typically taught at precollege or undergraduate level focuses heavily on the analysis of normally distributed data.

Students are taught to calculate the mean and the standard deviation (or variance), and sometimes higher moments, such as skew and kurtosis. However, these techniques have a serious drawback, in that the results can easily become distorted if the distribution has even relatively few far-flung outlying points.

NOTE

In Java performance, the outliers represent slow transactions and unhappy customers. We need to pay special attention to these points and avoid techniques that dilute the importance of outliers.

In [Figure 2-4](#) we can see a more realistic curve for the likely distribution of method (or transaction) times. It is clearly not a normal distribution.

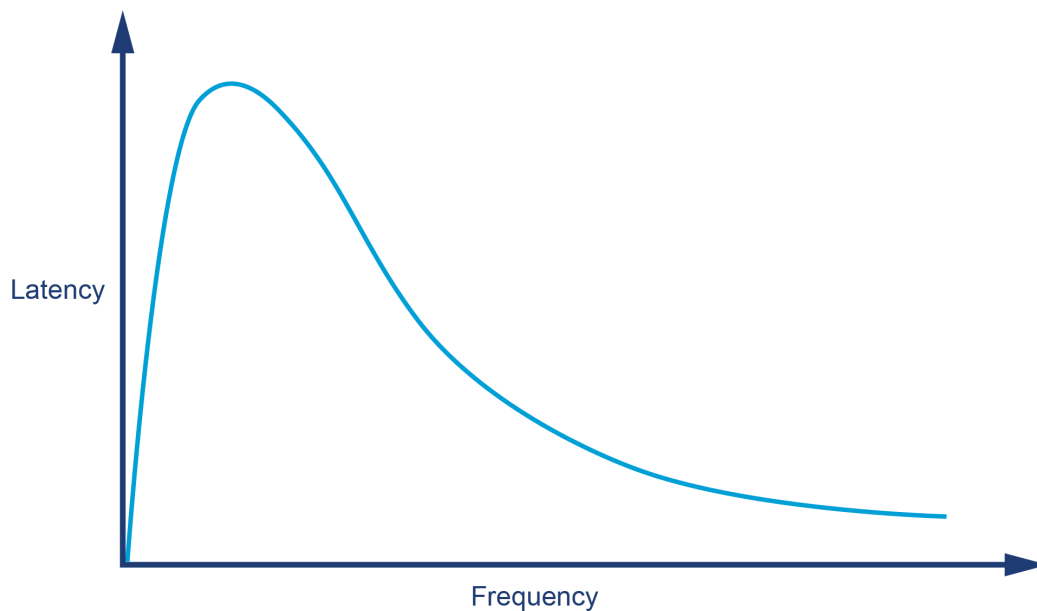


Figure 2-4. A more realistic view of the distribution of transaction times

The shape of the distribution in [Figure 2-4](#) shows something that we know intuitively about the JVM: it has “hot paths” where all the relevant code is already JIT-compiled, there are no GC cycles, and so on. These represent a best-case scenario (albeit a common one); there simply are no calls that are “a bit faster” due to random effects.

This violates a fundamental assumption of Gaussian statistics and forces us to consider distributions that are non-normal.

NOTE

For distributions that are non-normal, many “basic rules” of normally distributed statistics are violated. In particular, standard deviation/variance and other higher moments are basically useless.

To consider it from another viewpoint: unless a large number of customers are already complaining, it is unlikely that improving the average response time is a useful performance goal. For sure, doing so will improve the experience for everyone, but it is far more usual for a few disgruntled customers to be the cause of a latency tuning exercise. This implies that the outlier events are likely to be of more interest than the experience of the majority who are receiving satisfactory service.

One technique that is very useful for handling the non-normal, “long-tail” distributions the JVM produces is to use a modified scheme of percentiles. Remember that a distribution is a whole collection of points—a shape of data, and is not well-represented by a single number.

Instead of computing just the mean, which tries to express the whole distribution in a single result, we can use a sampling of the distribution at intervals. When used for normally distributed data, the samples are usually taken at regular intervals. However, a small adaptation allows the technique to be used more effectively for JVM statistics.

The modification is to use a sampling that takes into account the long-tail distribution by starting from the mean, then the 90th percentile, and then moving out logarithmically, as shown in the following method timing results. This means that we're sampling according to a pattern that better corresponds to the shape of the data:

```
50.0% level was 23 ns
90.0% level was 30 ns
99.0% level was 43 ns
99.9% level was 164 ns
99.99% level was 248 ns
99.999% level was 3,458 ns
99.9999% level was 17,463 ns
```

The samples show us that while the average time was 23 ns to execute a getter method, for 1 request in 1,000, the time was an order of magnitude worse, and for 1 request in 100,000, it was *two* orders of magnitude worse than average.

Long-tail distributions can also be referred to as *high dynamic range* (HDR) distributions. The dynamic range of an observable is usually defined as the maximum recorded value divided by the minimum (assuming the latter is nonzero).

Logarithmic percentiles are a useful simple tool for understanding the long tail. However, for more sophisticated analysis, we can use a public domain library for handling datasets with high dynamic range. The library, called HdrHistogram, is [available from GitHub](#). It was originally created by Gil Tene (Azul Systems), with additional work by Mike Barker and other contributors.

NOTE

A histogram is a way of summarizing data by using a finite set of ranges (called *buckets*) and displaying how often data falls into each bucket.

HdrHistogram is also available on Maven Central. At the time of writing, the current version is 2.1.12, and you can add it to your projects by adding this dependency stanza to *pom.xml*:

```
<dependency>
  <groupId>org.hdrhistogram</groupId>
  <artifactId>HdrHistogram</artifactId>
  <version>2.1.12</version>
</dependency>
```

Let's look at a simple example using HdrHistogram. This example takes in a file of numbers and computes the HdrHistogram for the difference between successive results:

```
public class BenchmarkWithHdrHistogram {
    private static final long NORMALIZER = 1_000_000;

    private static final Histogram HISTOGRAM
        = new Histogram(TimeUnit.MINUTES.toMicros(1), 2);

    public static void main(String[] args) throws Exception {
        final List<String> values = Files.readAllLines(Paths.get(args[0]));
        double last = 0;
        for (final String tVal : values) {
            double parsed = Double.parseDouble(tVal);
            double gcInterval = parsed - last;
            last = parsed;
            HISTOGRAM.recordValue((long)(gcInterval * NORMALIZER));
        }
        HISTOGRAM.outputPercentileDistribution(System.out, 1000.0);
    }
}
```

The output shows the times between successive garbage collections. As we'll see in Chapters [4](#) and [5](#), GC does not occur at regular intervals, and understanding the distribution of how frequently it occurs could be useful. Here's what the histogram plotter produces for a sample GC log:

Value	Percentile	TotalCount	1/(1-Percentile)
14.02	0.000000000000	1	1.00
1245.18	0.100000000000	37	1.11
1949.70	0.200000000000	82	1.25
1966.08	0.300000000000	126	1.43
1982.46	0.400000000000	157	1.67

...

```
28180.48 0.996484375000      368      284.44
28180.48 0.996875000000      368      320.00
28180.48 0.997265625000      368      365.71
36438.02 0.997656250000      369      426.67
36438.02 1.000000000000      369
#[Mean    =      2715.12, StdDeviation    =      2875.87]
#[Max     =      36438.02, Total count    =      369]
#[Buckets =           19, SubBuckets     =      256]
```

The raw output of the formatter is rather hard to analyze, but fortunately, the HdrHistogram project includes an [online formatter](#) that can be used to generate visual histograms from the raw output.

For this example, it produces output like that shown in [Figure 2-5](#).

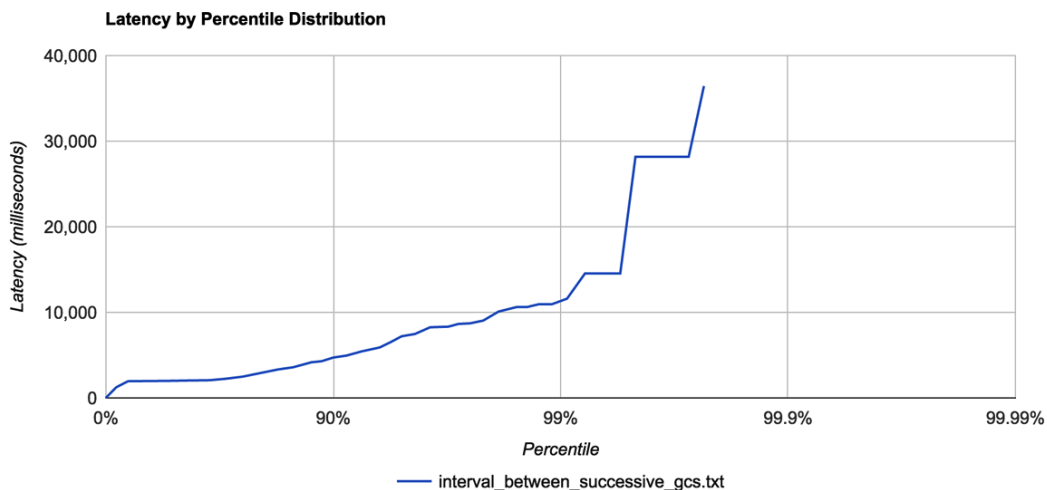


Figure 2-5. Example HdrHistogram visualization

For many observables that we wish to measure in Java performance tuning, the statistics are often highly non-normal, and HdrHistogram can be a very useful tool in helping to understand and visualize the shape of the data.

The tool can also help with detecting and understanding *coordinated omission*. This is a term that describes the phenomenon in which the measuring system inadvertently coordinates with the system being measured in a way that either incorrectly measures outliers or causes some requests to be skipped and not sent.⁵

Interpretation of Statistics

Empirical data and observed results do not exist in a vacuum, and it is quite common that one of the hardest jobs lies in interpreting the results we obtain from measuring our applications.

No matter what the problem is, it's always a people problem.

—Gerald Weinberg (attr)

Quite a few different problems are associated with interpretation. Let's start by taking a quick look at one notorious issue that frequently accompanies systematic error—spurious correlation.

Spurious Correlation

One of the most famous aphorisms about statistics is “correlation does not imply causation”—that is, just because two variables appear to behave similarly does not imply that there is an underlying connection between them.

This is a very important concept for a performance engineer to grasp and warrants a bit of unpacking. Wikipedia enumerates [four different options](#). For any two correlated events, A and B, there are these possible relationships:

- A causes B (direct causation).
- B causes A (reverse causation).
- A and B are both caused by C (common causation)
- There is no connection between A and B; the correlation is a coincidence.

The first two cases are relatively straightforward. The third case, common causation, is the situation when two variables are linked, but we draw an incorrect causal link. That is, it's a true correlation, not a spurious one, but that doesn't mean we can infer a causal relationship.

For example, in healthcare, breastfeeding babies in infancy correlates with higher IQ scores later in childhood. This is well-evidenced, but in this case, there's a third factor that is causal and drives the correlation—essentially, that higher rates of breastfeeding tend to occur with higher social class, which in turn leads to more time spent on encouraging a child's mental development.

In the most extreme examples of the fourth case, if a practitioner looks hard enough, then a correlation can be found between entirely unrelated measurements. For example, in [Figure 2-6](#) we can see that consumption of chicken in the US is well correlated with total import of crude oil.⁶

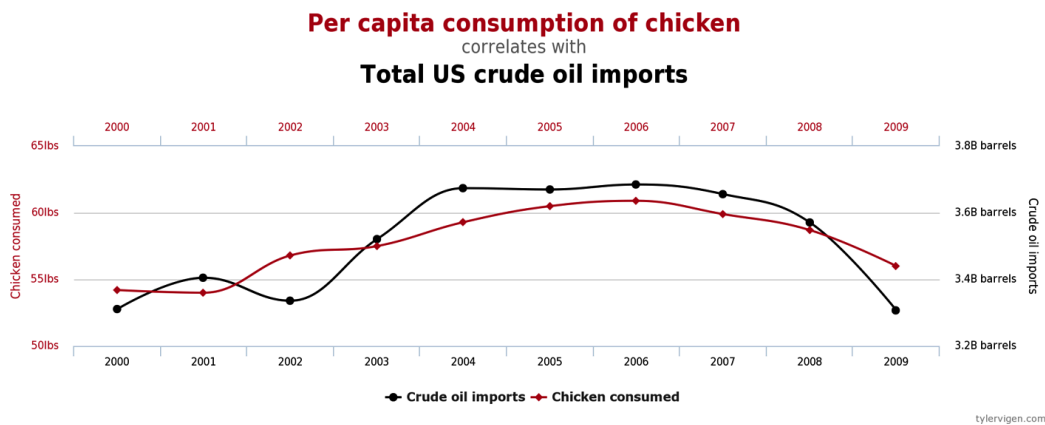


Figure 2-6. A completely spurious correlation (Vigen)

These numbers are clearly not causally related; there is no factor that drives both the import of crude oil and the eating of chicken. However, it isn't the absurd and ridiculous correlations that the practitioner needs to be wary of.

In [Figure 2-7](#), we see the revenue generated by video arcades correlated to the number of computer science PhDs awarded. It isn't too much of a stretch to imagine a sociological study that claimed a link between these observables, perhaps arguing that "stressed doctoral students were finding relaxation with a few hours of video games." These types of claim are depressingly common, despite no such common factor actually existing.

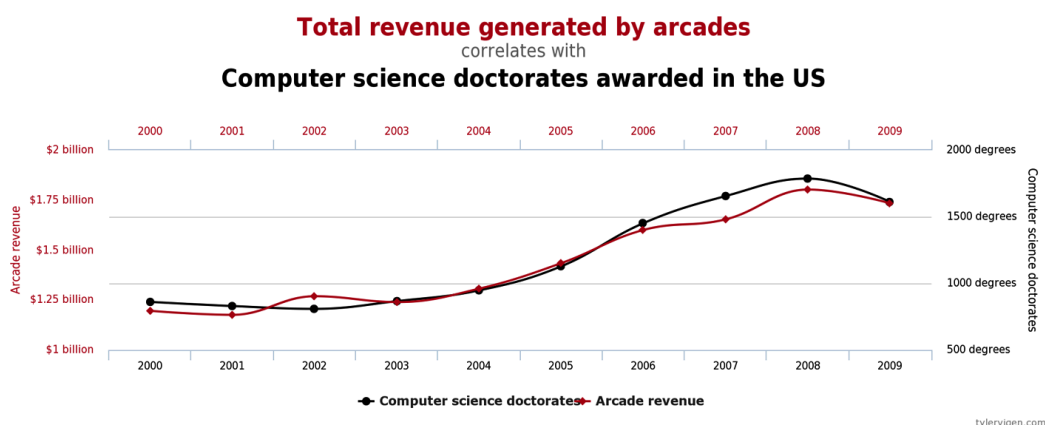


Figure 2-7. A less spurious correlation? (Vigen)

In the realm of the JVM and performance analysis, we need to be especially careful not to attribute a causal relationship between measurements based solely on correlation and that the connection "seems plausible."

*The first principle is that you must not fool yourself—and you are the easiest person to fool.*⁷

—Richard Feynman

In [Figure 2-8](#), we show an example memory allocation rate for a real Java application. This example is for a reasonably well-performing application.

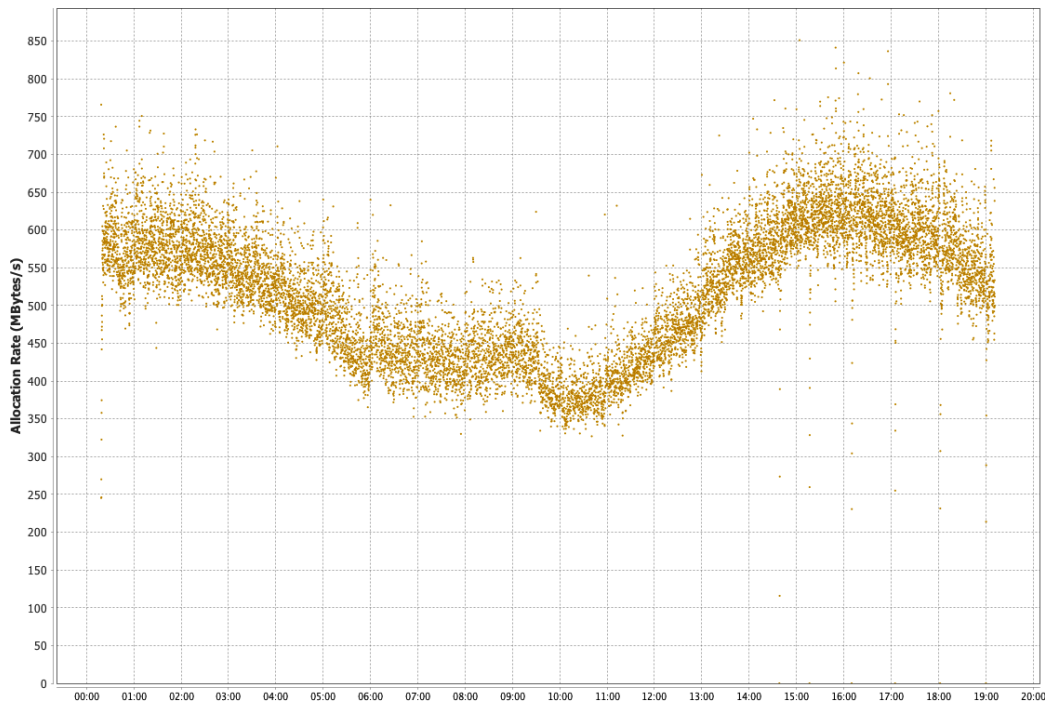


Figure 2-8. Example allocation rate

The interpretation of the allocation data is relatively straightforward, as there is a clear signal present. Over the time period covered (almost a day), allocation rates were basically stable between 350 and 700 MB per second. There is a downward trend starting approximately 5 hours after the JVM started up, and a clear minimum between 9 and 10 hours, after which the allocation rate starts to rise again.

These types of trends in observables are very common, as the allocation rate will usually reflect the amount of work an application is actually doing, and this will vary widely depending on the time of day. However, when we are interpreting real observables, the picture can rapidly become more complicated.

The Hat/Elephant Problem

This can lead to what is sometimes called the “hat/elephant” problem, after a passage in *The Little Prince* by Antoine de Saint-Exupéry. In the book, the narrator describes drawing, at age six, a picture of a boa con-

strictor that has eaten an elephant. However, as the view is external, the picture just resembles (at least to the ignorant eyes of the adults in the story) a slightly shapeless hat.

The metaphor stands as an admonition to the reader to have some imagination and to think more deeply about what you are really seeing, rather than just accepting a shallow explanation at face value.

The problem, as applied to software, is illustrated by [Figure 2-9](#). All we can initially see is a complex histogram of HTTP request-response times. However, just like the narrator of the book, if we can imagine or analyze a bit more, we can see that the complex picture is actually made up of several fairly simple pieces.

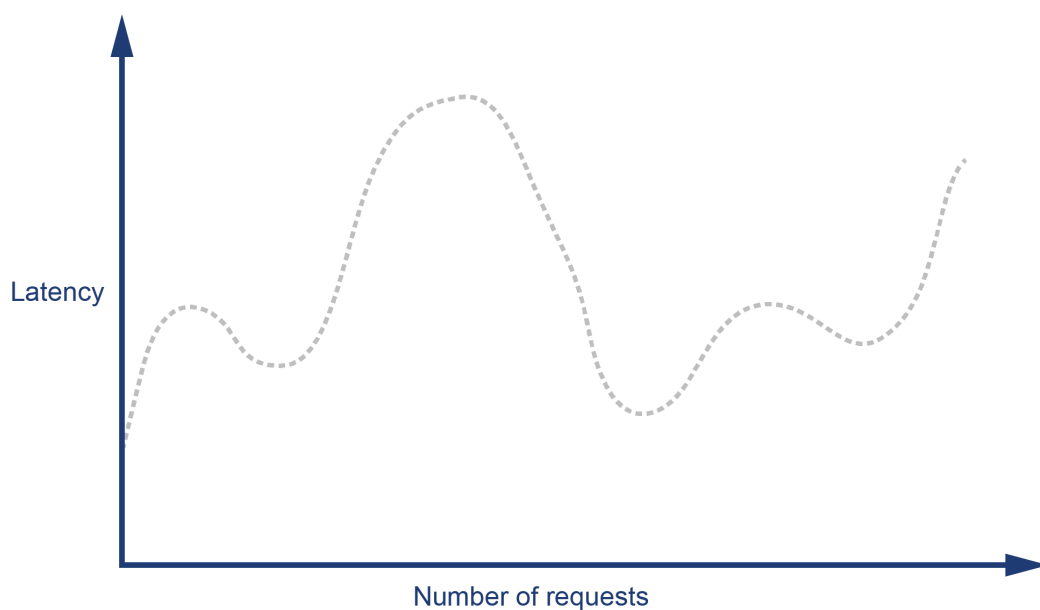


Figure 2-9. Hat or elephant eaten by a boa?

The key to decoding the response histogram is to realize that “web application responses” is a very general category, including successful requests (so-called 2xx responses), client errors (4xx, including the infamous 404 error), and server errors (5xx, especially 500 Internal Server Error).

Each type of response has a different characteristic distribution for response times. If a client makes a request for a URL that has no mapping (a 404), then the web server can immediately reply with a response. This means that the histogram for only client error responses looks more like [Figure 2-10](#).

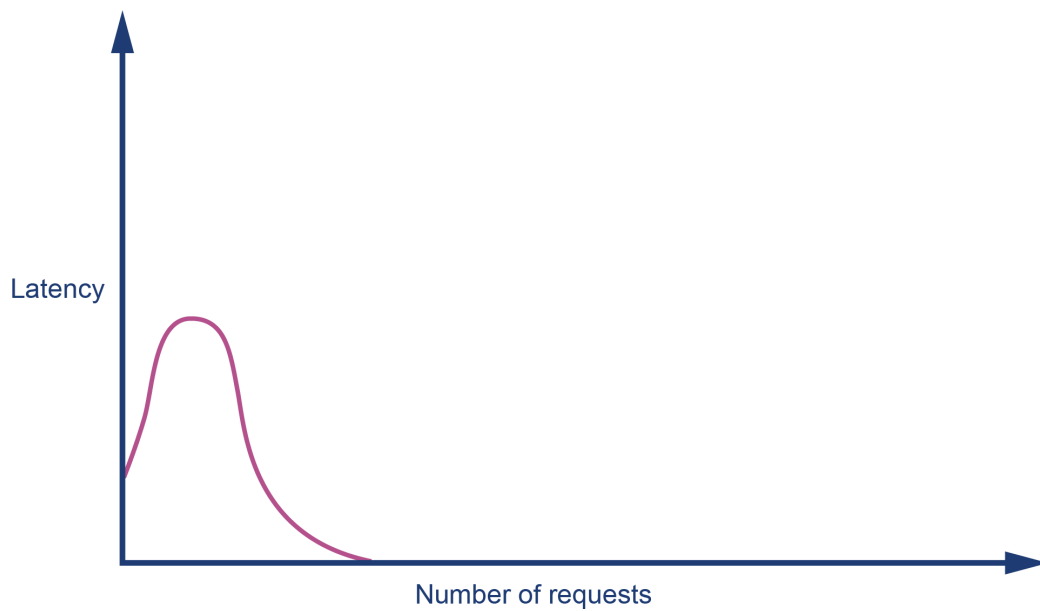


Figure 2-10. Client errors

By contrast, server errors often occur after a large amount of processing time has been expended (for example, due to backend resources being under stress or timing out). So, the histogram for server error responses might look like [Figure 2-11](#).

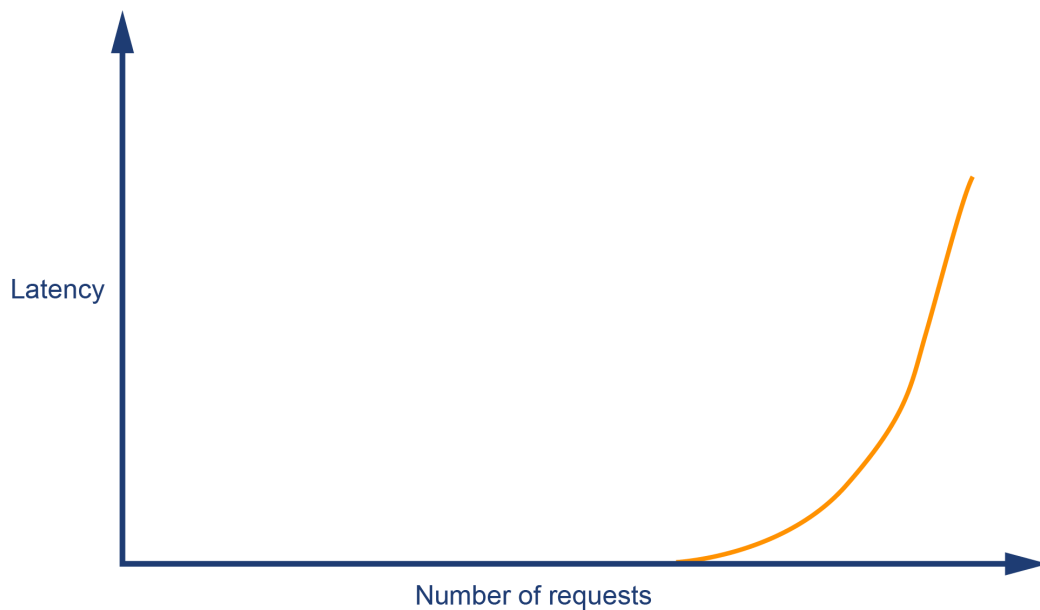


Figure 2-11. Server errors

The successful requests will have a long-tail distribution, but in reality, we can expect the response distribution to be “multimodal” and have several local maxima. An example is shown in [Figure 2-12](#) and represents the possibility that there could be two common execution paths through the application with quite different response times.

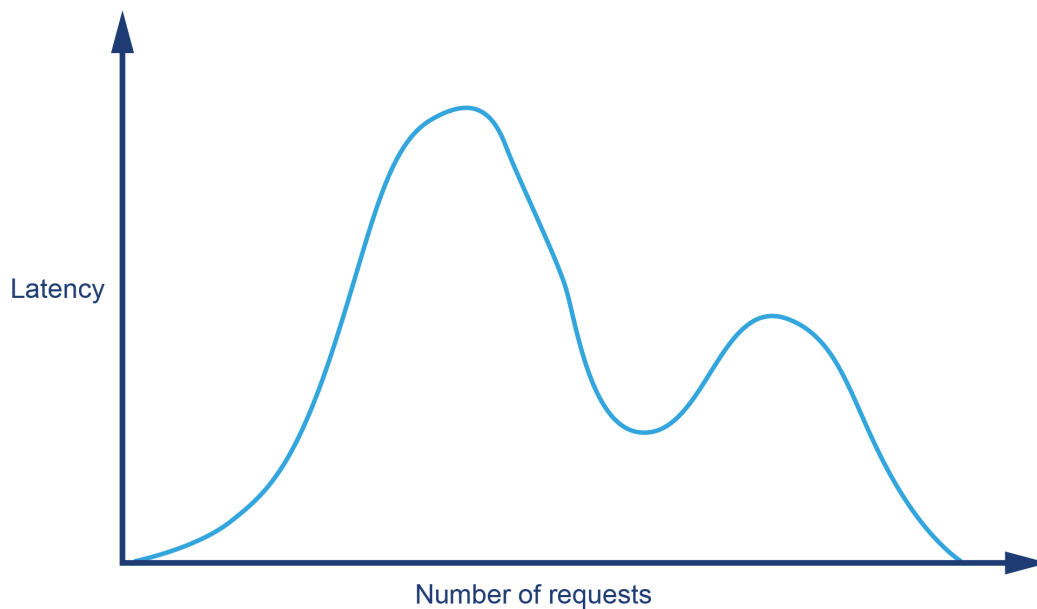


Figure 2-12. Successful requests

Combining these different types of responses into a single graph results in the structure shown in [Figure 2-13](#). We have rederived our original “hat” shape from the separate histograms.

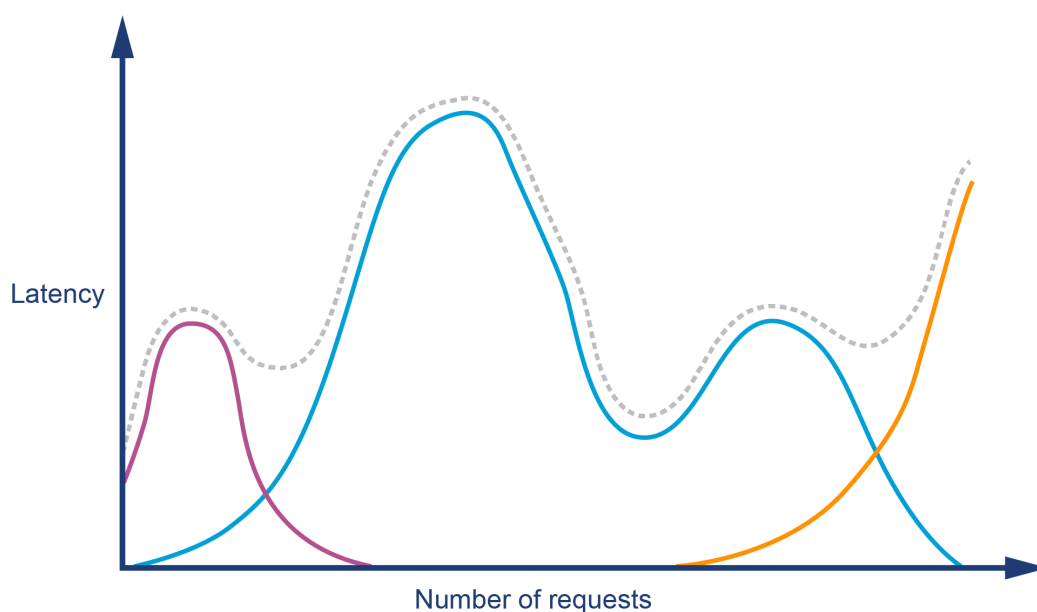


Figure 2-13. Hat or elephant revisited

The concept of breaking down a general observable into more meaningful sub-populations is a very useful one. It shows that we need to make sure that we understand our data and domain well enough before trying to infer conclusions from our results. We may well want to further break down our data into smaller sets; for example, the successful requests may have very different distributions for requests that are predominantly read, as opposed to requests that are updates or uploads.

The engineering team at PayPal has written extensively about its use of statistics and analysis; [they have a blog](#) that contains excellent resources. In particular, the piece [“Statistics for Software”](#) by Mahmoud Hashemi is

a great introduction to their methodologies and includes a version of the hat/elephant problem discussed earlier.⁸

Also worth mentioning is the “Datasaurus Dozen”—a collection of datasets that have the same basic statistics but wildly different appearances.⁹

Cognitive Biases and Performance Testing

Humans can be bad at forming accurate opinions quickly—even when faced with a problem where they can draw upon past experiences and similar situations.

A cognitive bias is a psychological effect that causes the human brain to draw incorrect conclusions. It is especially problematic because the person exhibiting the bias is usually unaware of it and may believe they are being rational.

Many of the antipatterns we observe in performance analysis (such as those in [Appendix B](#), which you might want to read in conjunction with this section) are caused, in whole or in part, by one or more cognitive biases that are, in turn, based on unconscious assumptions.

For example, with the *Blame Donkey* antipattern, if a component has caused several recent outages, the team may be biased to expect that same component to be the cause of any new performance problem. Any data that’s analyzed may be more likely to be considered credible if it confirms the idea that the Blame Donkey component is responsible.

The antipattern combines aspects of the biases known as confirmation bias and recency bias (a tendency to assume that whatever has been happening recently will keep happening).

NOTE

A single component in Java can behave differently from application to application depending on how it is optimized at runtime. To remove any pre-existing bias, it is important to look at the application as a whole.

Biases can be complementary or dual to each other. For example, some developers may be biased to assume that the problem is not software-related at all, and the cause must be the infrastructure the software is running on; this is common in the *Works for Me* antipattern, characterized by statements like: “This worked fine in UAT, so there must be a problem with the production kit.” The converse is to assume that every problem must be caused by software, because that’s the part of the system the developer knows about and can directly affect.

Let’s meet some of the most common biases that every performance engineer should look out for.

*Knowing where the trap is—that’s the first step in evading it.*¹⁰

—Duke Leto Atreides I

By recognizing these biases in ourselves, and others, we increase the chance of being able to do sound performance analysis and solve the problems in our systems.

Reductionist Thinking

The reductionist thinking cognitive bias is based on an analytical approach that presupposes that if you break a system into small enough pieces, you can understand it by understanding its constituent parts. Understanding each part means reducing the chance of incorrect assumptions being made.

The major problem with this view is simple to explain—in complex systems, it just isn’t true. Nontrivial software (or physical) systems almost always display emergent behavior, where the whole is greater than a simple summation of its parts would indicate.

Confirmation Bias

Confirmation bias can lead to significant problems when it comes to performance testing or attempting to look at an application subjectively. A confirmation bias is introduced, usually not intentionally, when a poor test set is selected or results from the test are not analyzed in a statistically sound way. Confirmation bias is quite hard to counter, because there are often strong motivational or emotional factors at play (such as someone in the team trying to prove a point).

Consider an antipattern such as *Distracted by Shiny*, where a team member is looking to bring in the latest and greatest NoSQL database. They run some tests against data that isn't like production data, because representing the full schema is too complicated for evaluation purposes.

They quickly prove that on a test set the NoSQL database produces superior access times on their local machine. The developer has already told everyone this would be the case, and on seeing the results, they proceed with a full implementation. There are several antipatterns at work here, all leading to new, unproved assumptions in the new library stack.

Fog of War (Action Bias)

The fog of war bias usually manifests itself during outages or situations where the system is not performing as expected and the team is under pressure. Some common causes include:

- Changes to infrastructure that the system runs on, perhaps without notification or realizing there would be an impact
- Changes to libraries that the system is dependent on
- A strange bug or race condition that manifests itself, but only on busy days

In a well-maintained application with sufficient observability tooling, these should generate clear signals that will lead the support team to the cause of the problem.

However, too many applications have not tested failure scenarios and lack appropriate tooling. Under these circumstances even experienced engineers can fall into the trap of needing to feel that they're doing something to resolve the outage and mistaking motion for velocity—the “fog of war” descends.

At this time, many of the human elements discussed in this chapter can come into play if participants are not systematic about their approach to the problem.

For example, an antipattern such as *Blame Donkey* can shortcut a full investigation and lead the production team down a particular path of investigation—often missing the bigger picture. Similarly, the team may be tempted to break the system down into its constituent parts and look through the code at a low level without first establishing in which subsystem the problem truly resides.

Risk Bias

Humans are naturally risk averse and resistant to change. Mostly this is because people have seen examples of how change can cause things to go wrong—this leads them to attempt to avoid that risk. This can be incredibly frustrating when taking small, calculated risks could move the product forward. Much of this risk aversion arises from teams that are reluctant to make changes that might modify the performance profile of the application.

We can reduce this risk bias significantly by having a robust set of unit tests and production regression tests. The performance regression tests are a great place to link in the system’s nonfunctional requirements and ensure that the concerns the NFRs represent are reflected in the regression tests.

However, if either of these is not sufficiently trusted by the team, change becomes extremely difficult, and the risk factor is not controlled. This bias often manifests in a failure to learn from application problems (including service outages) and implement appropriate mitigation.

Summary

When you are evaluating performance results, it is essential to handle the data appropriately and avoid falling into unscientific and subjective thinking. This includes avoiding the statistical pitfalls of relying upon Gaussian models when they are not appropriate.

In this chapter, we have met some different types of performance tests, testing best practices, and human problems that are native to performance analysis.

In the next chapter, we’re going to move on to an overview of the JVM, introducing the basic subsystems, the lifecycle of a “classic” Java application, and a first look at monitoring and tooling.

¹ The term was popularized by the book *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*, by William J. Brown, Raphael C. Malveau, Hays W. McCormick III, and Thomas J. Mowbray (New York: Wiley, 1998).

² Mary Walton, *The Deming Management Method* (Mercury Books, 1989).

- ³ Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. “Statistically Rigorous Java Performance Evaluation,” *ACM SIGPLAN Notices*, vol. 42, iss. 10 (October 2007): 57–76.
- ⁴ John H. McDonald, *Handbook of Biological Statistics*, 3rd ed. (Baltimore, MD: Sparky House Publishing, 2014).
- ⁵ For more on this, see Ivan Prisyazhynny, “On Coordinated Omission,” Scylla, April 22, 2021, <https://oreil.ly/VoW98>.
- ⁶ The spurious correlations in this section come from Tyler Vigen’s site and are reused here with permission under CC BY 4.0. If you enjoy them, a book with many more amusing examples is available from his website.
- ⁷ Richard Feynman and Ralph Leighton, *Surely You’re Joking, Mr. Feynman!* (W.W. Norton, 1985).
- ⁸ Mahmoud Hashemi wrote this article when working for PayPal, but this is now attributed to the Paypal Tech Blog Team.
- ⁹ Justin Matejka and George Fitzmaurice, “Same Stats, Different Graphs: Generating Datasets with Varied Appearance and Identical Statistics through Simulated Annealing,” *ACM SIGCHI Conference on Human Factors in Computing Systems 2017*, Denver (2017).
- ¹⁰ Frank Herbert, *Dune* (Chilton Books, 1965).